

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON (8 SEMANAS)

NIVEL 2 (INTERMEDIO)

## Curso 2: Algoritmos Avanzados y Estructuras de Datos

«Optimización de Memoria, Notación Big-O, Pilas, Colas, Búsqueda Binaria, Grafos y Programación Dinámica»

Manual completo del curso compilado oficialmente. Incluye el desglose de las 8 clases semanales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor: William Rodríguez (Wisrovi)

Total Clases: 8 Semanas | Python: 3.10+ | Licencia: MIT

## Acerca del Autor y Mentor



### William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



**GitHub:** [github.com/wisrovi](https://github.com/wisrovi)



**LinkedIn:** [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



**DockerHub:** [hub.docker.com/u/wisrovi](https://hub.docker.com/u/wisrovi)



**Website:** [wisrovi.dev](https://wisrovi.dev)

## Metodología: La Regla de la Bicicleta

Aprender a programar es pedalear en VS Code: la experimentación práctica diaria es la clave del dominio técnico.

## Plan de Estudios de las 8 Semanas

| Semana   | Unidad Temática                                                 | Metáfora                                                |
|----------|-----------------------------------------------------------------|---------------------------------------------------------|
| CLASE 01 | Clase 01: Análisis de Complejidad y Notación Big-O              | «Medir el Rendimiento de un Algoritmo a Medida que ...» |
| CLASE 02 | Clase 02: Pilas (Stacks) y Colas (Queues) con collections.deque | «Pilas LIFO como Platos Apilados y Colas FIFO como ...» |
| CLASE 03 | Clase 03: Tablas Hash y Conjuntos (Sets) para Búsqueda O(1)     | «Tablas Hash como un Fichero con Índice Alfabético ...» |
| CLASE 04 | Clase 04: Algoritmos de Búsqueda: Lineal vs Binaria O(log n)    | «Búsqueda Binaria como Buscar una Palabra en el Dic...» |
| CLASE 05 | Clase 05: Algoritmos de Ordenamiento: QuickSort y MergeSort     | «Ordenar Barajas de Cartas con Divide y Vencerás...»    |
| CLASE 06 | Clase 06: Árboles Binarios de Búsqueda (BST) y Recorridos       | «Árboles como Organigramas Jerárquicos con Ramas Iz...» |
| CLASE 07 | Clase 07: Grafos, Matrices de Adyacencia y Recorridos BFS/DFS   | «Grafos como Redes de Ciudades y Rutas de Vuelo...»     |
| CLASE 08 | Clase 08: Recursividad y Programación Dinámica con Memoización  | «Programación Dinámica como Recordar el Pasado para...» |

## Clase 01: Análisis de Complejidad y Notación Big-O

La notación Big-O describe cómo escala el tiempo de ejecución y el uso de memoria de un algoritmo.

### ☀ Metáfora: Medir el Rendimiento de un Algoritmo a Medida que Crece la Entrada

Big-O es como calcular cuánta gasolina consumirá un camión de carga según el número de kilómetros y peso.

### Principios Teóricos

Nos enfocamos en el peor caso (Worst-case scenario) y descartamos constantes y términos de menor orden.

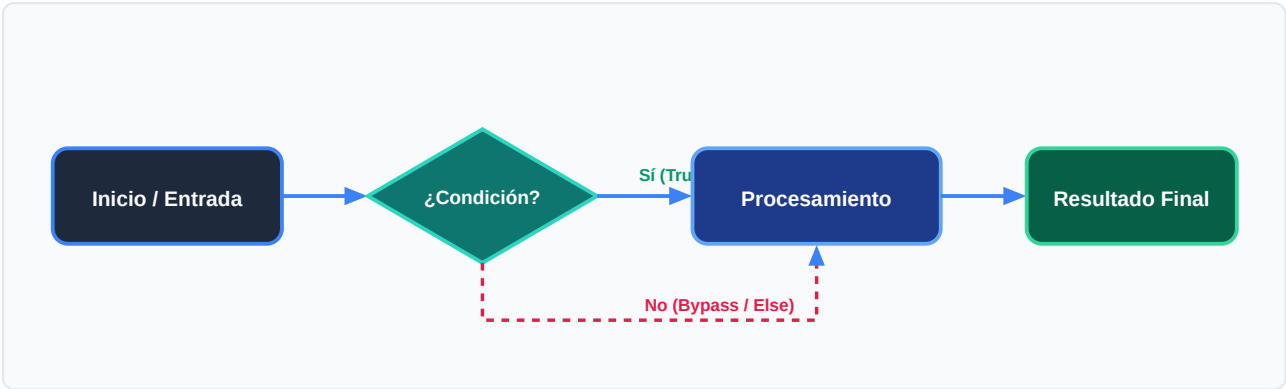
Un algoritmo  $O(n)$  sobre 1 millón de elementos tarda milisegundos; un  $O(n^2)$  puede tardar horas.

### ⚡ Regla de Oro

Evita los bucles anidados innecesarios para prevenir la degradación a  $O(n^2)$ .

## Arquitectura y Movimiento de Datos

Comparativa de curvas de crecimiento asintótico frente a volumen de datos  $n$ .



| Fase       | Acción del Intérprete                                            | Estado Memoria              |
|------------|------------------------------------------------------------------|-----------------------------|
| 1. Inicio  | Cálculo de operaciones elementales.                              | Función matemática $T(n)$ . |
| 2. Proceso | Eliminación de coeficientes constantes.                          | Simplificación algebraica.  |
| 3. Salida  | Benchmarking experimental con <code>time.perf_counter()</code> . | Validación empírica en ms.  |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import time

def acceso_ol(lista: list, idx: int):
    return lista[idx] # O(1)

def busqueda_on(lista: list, target: int):
    for item in lista: # O(n)
        if item == target:
            return True
    return False

datos = list(range(1_000_000))
print("O(1) Acceso:", acceso_ol(datos, 500_000))
print("O(n) Búsqueda:", busqueda_on(datos, 999_999))
```

### ⚠ Gotcha Frecuente

Usar 'if x in lista:' dentro de un bucle for convierte tu código silenciosamente en  $O(n^2)$ .

#### ❌ Antipatrón

for elem in lista\_a: if elem in lista\_b: # ❌ 'in' en lista es  $O(n)$ , total  $O(n^2)$  comunes.append(elem)

#### ✅ Patrón Correcto

set\_b = set(lista\_b) #  $O(n)$  for elem in lista\_a: if elem in set\_b: # ✅ 'in' en set es  $O(1)$ , total  $O(n)$  comunes.append(elem)

## Clase 02: Pilas (Stacks) y Colas (Queues) con collections.deque

Las pilas y colas son estructuras de datos lineales que restringen la inserción y extracción según una disciplina estricta.

### ☀ Metáfora: Pilas LIFO como Platos Apilados y Colas FIFO como la Fila del Supermercado

Una pila es como una torre de platos (el último que pones es el primero que lavas); una cola es la fila del banco (el primero en llegar es el primero en ser atendido).

### Principios Teóricos

Una lista de Python como cola es ineficiente: `lista.pop(0)` es  $O(n)$  porque desplaza todos los elementos en memoria.

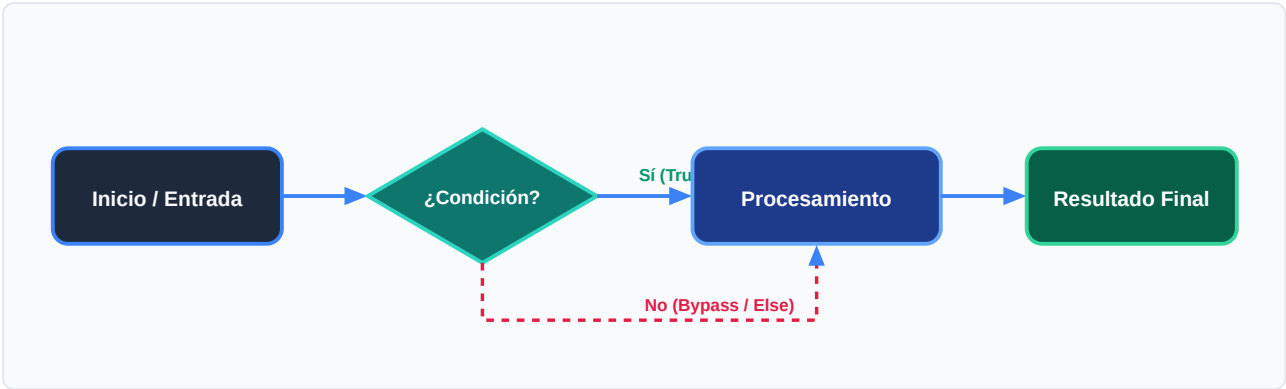
`collections.deque` implementa una lista doblemente enlazada con inserción y extracción  $O(1)$  en ambos extremos.

### ⚡ Regla de Oro

Nunca uses `lista.pop(0)` para colas; usa siempre `collections.deque().popleft()`.

## Arquitectura y Movimiento de Datos

Mecanismo de inserción y extracción LIFO (Stack) vs FIFO (Queue).



| Fase       | Acción del Intérprete                   | Estado Memoria                  |
|------------|-----------------------------------------|---------------------------------|
| 1. Inicio  | Inserción de elementos (append / push). | Elemento en el extremo derecho. |
| 2. Proceso | Operación de consulta del tope (peek).  | Lectura sin extracción.         |
| 3. Salida  | Verificación de estructura vacía.       | Longitud 0 confirmada.          |



## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
from collections import deque\n\n# 1. Pila (Stack LIFO)\ndef balanceado(expr: str) -> bool:\n    pila = []\n    mapa = {")": "(", "}": "{", "]": "["}\n    for char in expr:\n        if char in\n            mapa.values(): pila.append(char)\n        elif char in mapa:\n            if not pila or\n                pila.pop() != mapa[char]: return False\n    return len(pila) == 0\n\n# 2. Cola (Queue\nFIFO)\ncola = deque(["Ticket 1", "Ticket 2", "Ticket 3"])\ncola.append("Ticket\n4")\nprint("Atendido:", cola.popleft()) # Ticket 1\nprint("Es valido:", balanceado("{[()]})")
```

### ⚠ Gotcha Frecuente

Usar `list.pop(0)` en colas con miles de elementos colapsa el rendimiento de la CPU.

#### ❌ Antipatrón

```
cola = []\ncola.append(x)\nprimero = cola.pop(0) # ❌\n# 0(n) movimiento de bloques en memoria
```

#### ✅ Patrón Correcto

```
from collections import deque\ncola = deque()\ncola.append(x)\nprimero = cola.popleft() # ✅\n# 0(1) instantáneo
```

## Clase 03: Tablas Hash y Conjuntos (Sets) para Búsqueda O(1)

Las tablas hash convierten una clave arbitraria en un índice numérico mediante una función matemática de hashing.

### ☀ Metáfora: Tablas Hash como un Fichero con Índice Alfabético Instantáneo

Es como un conserje de hotel que sabe instantáneamente el casillero de cada huésped con solo mirar su apellido.

### Principios Teóricos

Las colisiones ocurren cuando dos claves distintas generan el mismo hash; CPython usa open addressing con perturbación.

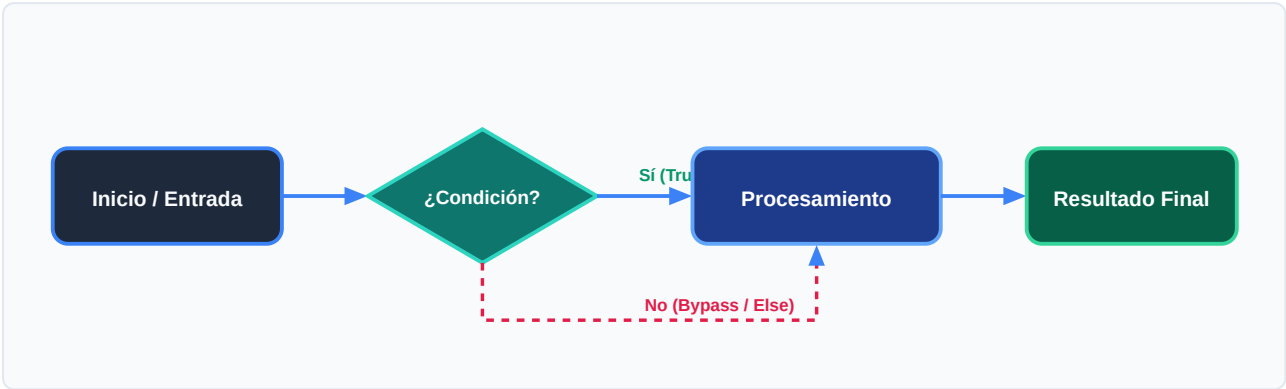
Las operaciones de inserción, búsqueda y eliminación en sets y dicts tienen complejidad temporal promedio O(1).

### ⚡ Regla de Oro

Cualquier objeto que uses como clave de diccionario o elemento de set debe implementar `__hash__` e inmutabilidad.

## Arquitectura y Movimiento de Datos

Proceso de hashing, resolución de buckets y deduplicación.



| Fase       | Acción del Intérprete                   | Estado Memoria               |
|------------|-----------------------------------------|------------------------------|
| 1. Inicio  | Invocación de hash(objeto).             | Firma hash de 64 bits.       |
| 2. Proceso | Mapeo al índice del arreglo de buckets. | Posición en tabla calculada. |
| 3. Salida  | Inserción o retorno del valor.          | Estado de tabla actualizado. |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
def two_sum(nums: list[int], target: int) -> tuple[int, int]:
    vistos = {} # mapa: valor -> indice
    for i, num in enumerate(nums):
        complemento = target - num
        if complemento in vistos:
            return (vistos[complemento], i)
        vistos[num] = i
    return (-1, -1)

indices = two_sum([2, 7, 11, 15], 9)
print("Índices que suman 9:", indices) # (0, 1)
```

### ⚠ Gotcha Frecuente

Intentar usar una lista mutable como clave de diccionario o elemento de set genera `TypeError: unhashable type`.

#### ❌ Antipatrón

```
mi_dict = {} mi_dict[[1, 2]] = 'valor' # ❌ TypeError:
unhashable type: 'list'
```

#### ✅ Patrón Correcto

```
mi_dict = {} mi_dict[(1, 2)] = 'valor' # ✅ Tupla
immutable hashable
```

## Clase 04: Algoritmos de Búsqueda: Lineal vs Binaria $O(\log n)$

Buscar elementos en grandes volúmenes de datos requiere algoritmos más inteligentes que la simple inspección secuencial.

### ☀ Metáfora: Búsqueda Binaria como Buscar una Palabra en el Diccionario Dividiendo a la Mitad

Para encontrar la página 500 de un libro de 1000 páginas, lo abres a la mitad exacta y descartas 500 páginas de golpe.

### Principios Teóricos

Búsqueda Lineal  $O(n)$ : Inspecciona uno a uno. Funciona en listas desordenadas.

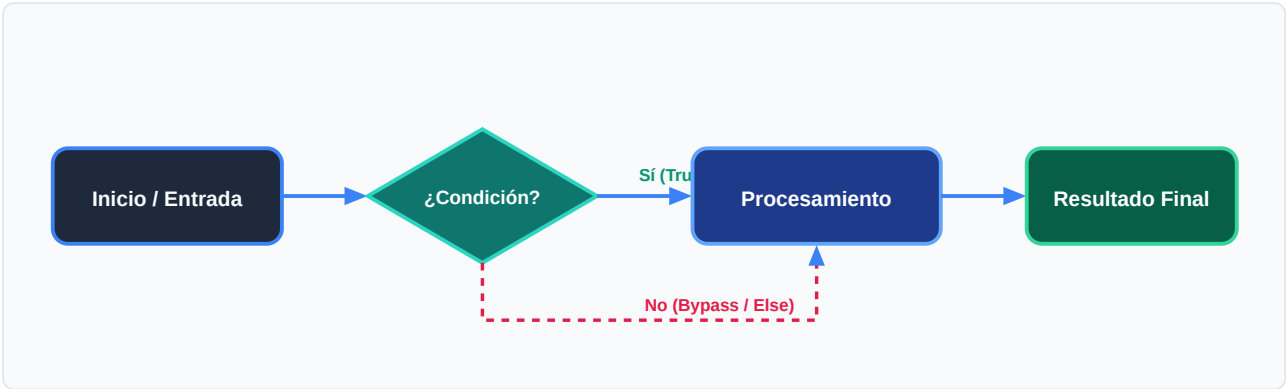
Búsqueda Binaria  $O(\log n)$ : Requiere que la lista esté estrictamente ordenada. En 1 millón de elementos tarda máximo 20 pasos.

### ⚡ Regla de Oro

La búsqueda binaria solo es válida sobre arreglos previamente ordenados.

## Arquitectura y Movimiento de Datos

Reducción del espacio de búsqueda mediante punteros left, right y mid.



| Fase       | Acción del Intérprete                                                          | Estado Memoria            |
|------------|--------------------------------------------------------------------------------|---------------------------|
| 1. Inicio  | Inicialización de punteros: $\text{left} = 0, \text{right} = \text{len} - 1$ . | Ventana completa activa.  |
| 2. Proceso | Cálculo del punto medio: $\text{mid} = (\text{left} + \text{right}) // 2$ .    | Puntero mid evaluado.     |
| 3. Salida  | Ajuste de límites o retorno del índice.                                        | Elemento encontrado o -1. |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import bisect

def busqueda_binaria(arr: list[int], target: int) -> int:
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

datos = [10, 20, 30, 40, 50, 60, 70, 80]
idx = busqueda_binaria(datos, 60)
print("Índice de 60:", idx) # 5
print("Índice con bisect_left:", bisect.bisect_left(datos, 60))
```

### ⚠ Gotcha Frecuente

Escribir `while left < right` en lugar de `left <= right` omite evaluar el último elemento restante.

#### ❌ Antipatrón

`while left < right: # ❌ Puede fallar si el target está en el último elemento`

#### ✅ Patrón Correcto

`while left <= right: # ✅ Evalúa todos los casos correctamente`

## Clase 05: Algoritmos de Ordenamiento: QuickSort y MergeSort

El ordenamiento es la base de la optimización en ciencias de la computación.

### ☀ Metáfora: Ordenar Barajas de Cartas con Divide y Vencerás

QuickSort elige un elemento pivote y separa las cartas en dos montones: menores a la izquierda, mayores a la derecha.

### Principios Teóricos

Algoritmos cuadráticos  $O(n^2)$  como Bubble o Insertion Sort son lentos para grandes volúmenes.

QuickSort y MergeSort tienen complejidad promedio  $O(n \log n)$ .

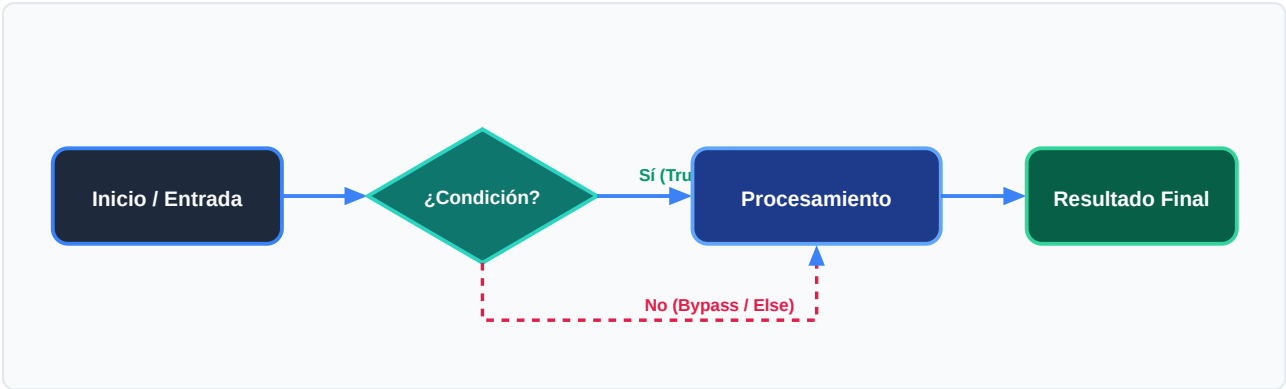
### ⚡ Regla de Oro

En producción, usa siempre el método `.sort()` de Python, que implementa Timsort (híbrido optimizado en C).



## Arquitectura y Movimiento de Datos

Particionamiento recursivo alrededor de un pivote.



| Fase       | Acción del Intérprete                  | Estado Memoria            |
|------------|----------------------------------------|---------------------------|
| 1. Inicio  | Caso base: len(arr) <= 1 retorna arr.  | Sublista trivial.         |
| 2. Proceso | Elección de pivote (elemento central). | Pivote seleccionado.      |
| 3. Salida  | Llamadas recursivas y concatenación.   | Lista combinada ordenada. |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
def quicksort(arr: list[int]) -> list[int]:
    if len(arr) <= 1:
        return arr
    pivote = arr[len(arr) // 2]
    menores = [x for x in arr if x < pivote]
    iguales = [x for x in arr if x == pivote]
    mayores = [x for x in arr if x > pivote]
    return quicksort(menores) + iguales + quicksort(mayores)

desordenados = [38, 27, 43, 3, 9, 82, 10]
print("Ordenados:", quicksort(desordenados))
```

### ⚠ Gotcha Frecuente

Elegir siempre el primer elemento como pivote en una lista que ya está ordenada genera  $O(n^2)$ .

#### ❌ Antipatrón

pivote = arr[0] # ❌ Degrada a  $O(n^2)$  si la lista ya viene ordenada

#### ✅ Patrón Correcto

pivote = arr[len(arr) // 2] # ✅ Pivote central o aleatorio

## Clase 06: Árboles Binarios de Búsqueda (BST) y Recorridos

Los árboles binarios organizan la información de manera jerárquica para permitir búsquedas e inserciones rápidas.

### ☀ Metáfora: Árboles como Organigramas Jerárquicos con Ramas Izquierda y Derecha

Un árbol BST es como un árbol genealógico donde a la izquierda van los números menores y a la derecha los mayores.

### Principios Teóricos

Propiedad BST: Para cada nodo, todo valor en su subárbol izquierdo es menor, y en su subárbol derecho es mayor.

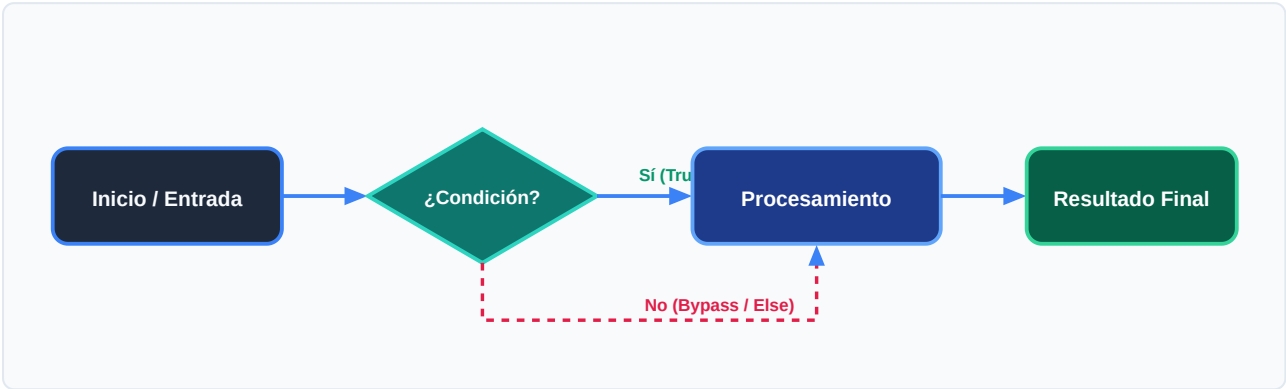
El recorrido in-order (Izquierda -> Raíz -> Derecha) extrae los elementos ordenados de menor a mayor.

### ⚡ Regla de Oro

Un árbol desbalanceado degenera en una lista enlazada  $O(n)$ ; los árboles balanceados mantienen  $O(\log n)$ .

## Arquitectura y Movimiento de Datos

Estructura de nodos en memoria y bifurcación izquierda/derecha.



| Fase       | Acción del Intérprete                             | Estado Memoria                |
|------------|---------------------------------------------------|-------------------------------|
| 1. Inicio  | Creación del nodo raíz con valor y punteros None. | Raíz instanciada.             |
| 2. Proceso | Inserción recursiva comparando valor < nodo.val.  | Navegación al hijo izquierdo. |
| 3. Salida  | Recorrido in-order para lectura ordenada.         | Lista ordenada generada.      |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class Nodo:
    def __init__(self, val: int):
        self.val = val
        self.izq = None
        self.der = None

def insertar(raiz: Nodo, val: int) -> Nodo:
    if not raiz: return Nodo(val)
    if val < raiz.val: raiz.izq = insertar(raiz.izq, val)
    else: raiz.der = insertar(raiz.der, val)
    return raiz

def in_order(raiz: Nodo, res: list):
    if raiz:
        in_order(raiz.izq, res)
        res.append(raiz.val)
        in_order(raiz.der, res)

raiz = None
for num in [50, 30, 70, 20, 40, 60, 80]:
    raiz = insertar(raiz, num)

elementos = []
in_order(raiz, elementos)
print("Recorrido In-Order:", elementos)
```

### ⚠ Gotcha Frecuente

Olvidar validar if not nodo antes de acceder a nodo.izq produce AttributeError: 'NoneType' object has no attribute.

#### ❌ Antipatrón

```
def buscar(nodo, val): if nodo.val == val: return True
# ❌ Falla si nodo es None
```

#### ✅ Patrón Correcto

```
def buscar(nodo, val): if not nodo: return False # ✅
Caso base de seguridad if nodo.val == val: return True
```

## Clase 07: Grafos, Matrices de Adyacencia y Recorridos BFS/DFS

Los grafos modelan relaciones complejas de red entre entidades (redes sociales, mapas, dependencias).

### ☀ Metáfora: Grafos como Redes de Ciudades y Rutas de Vuelo

Un grafo es un mapa de aeropuertos (nodos) conectados por vuelos (aristas).

### Principios Teóricos

BFS (Breadth-First Search) explora por capas concéntricas usando una Cola FIFO; encuentra el camino más corto.

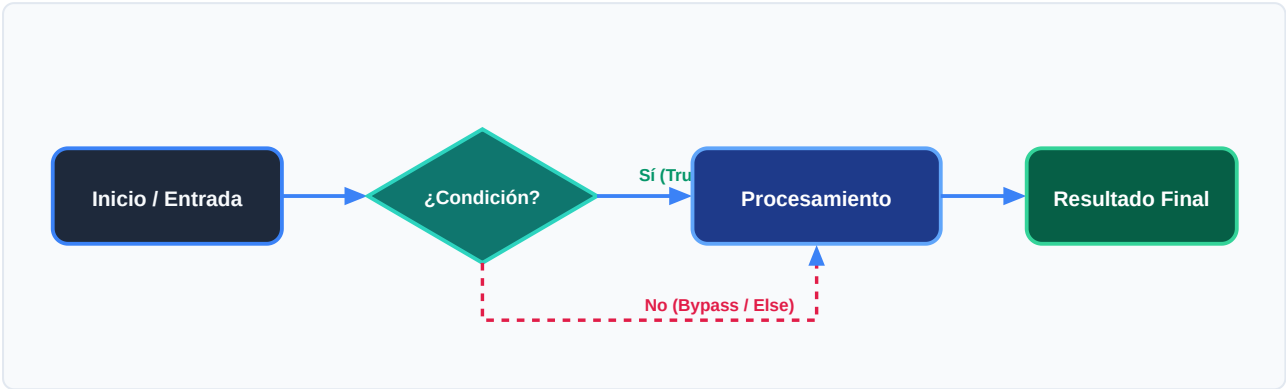
DFS (Depth-First Search) explora hasta el fondo de cada rama usando una Pila o recursión.

### ⚡ Regla de Oro

En grafos con ciclos, mantén siempre un conjunto 'visitados = set()' para evitar bucles infinitos.

## Arquitectura y Movimiento de Datos

Exploración por niveles con BFS vs exploración profunda con DFS.



| Fase       | Acción del Intérprete                                     | Estado Memoria                         |
|------------|-----------------------------------------------------------|----------------------------------------|
| 1. Inicio  | Representación del grafo como lista de adyacencia (dict). | Grafo en memoria.                      |
| 2. Proceso | Inicialización de cola FIFO con el nodo de origen.        | Cola = [inicio], visitados = {inicio}. |
| 3. Salida  | Fin de exploración tras vaciar la cola.                   | Árbol de expansión BFS obtenido.       |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
from collections import deque

grafo = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"]
}

def bfs(grafo: dict, inicio: str) -> list[str]:
    visitados = {inicio}
    cola = deque([inicio])
    recorrido = []
    while cola:
        nodo = cola.popleft()
        recorrido.append(nodo)
        for vecino in grafo.get(nodo, []):
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)
    return recorrido

print("Recorrido BFS:", bfs(grafo, "A"))
```

### ⚠ Gotcha Frecuente

No registrar los nodos en visitados provoca un RecursionError o bucle infinito.

#### ❌ Antipatrón

```
def dfs(nodo): for v in grafo[nodo]: dfs(v) # ❌ Sin control de visitados en grafo cíclico
```

#### ✅ Patrón Correcto

```
def dfs(nodo, visitados=None): if visitados is None:
    visitados = set() visitados.add(nodo) for v in
    grafo[nodo]: if v not in visitados: dfs(v, visitados) #
    ✅ Seguro
```



## Clase 08: Recursividad y Programación Dinámica con Memoización

La programación dinámica es una técnica de optimización que almacena los resultados de subproblemas ya resueltos.

### ☀ Metáfora: Programación Dinámica como Recordar el Pasado para no Resolverlo Dos Veces

Si te pregunto cuánto es  $1+1+1+1$ , cuentas 4. Si añado otro  $+1$ , sabes que es 5 porque recordaste el 4 anterior.

### Principios Teóricos

Recursión simple (Naive): Resuelve los mismos subproblemas miles de veces (explosión exponencial  $O(2^n)$ ).

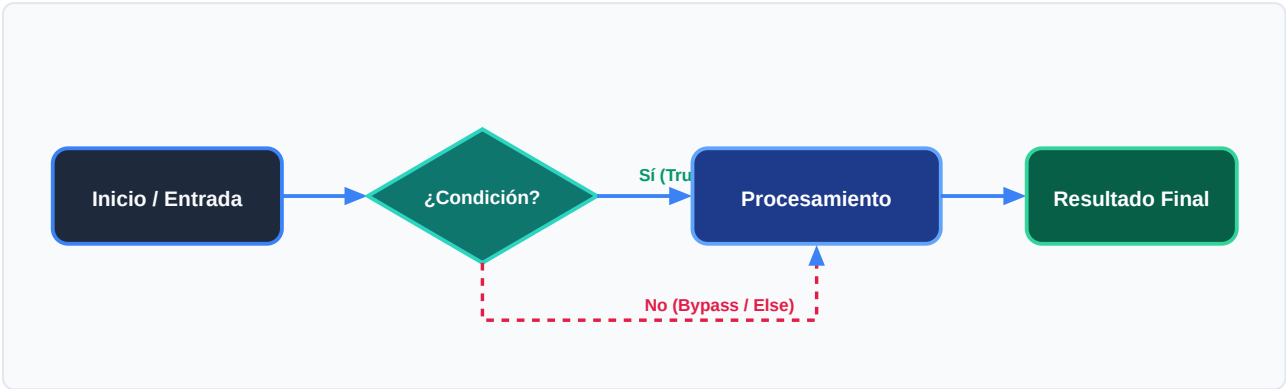
Memoización (Top-Down): Guarda en una tabla hash el resultado de cada llamada de función.

### ⚡ Regla de Oro

En Python, decora tus funciones recursivas puras con `@functools.lru_cache(maxsize=None)`.

## Arquitectura y Movimiento de Datos

Poda del árbol de recursión mediante caché de resultados.



| Fase       | Acción del Intérprete                   | Estado Memoria                       |
|------------|-----------------------------------------|--------------------------------------|
| 1. Inicio  | Llamada con argumentos n.               | Búsqueda en el diccionario de caché. |
| 2. Proceso | Cache Hit: Retorno instantáneo $O(1)$ . | Subárbol de cálculo omitido.         |
| 3. Salida  | Almacenamiento en caché y retorno.      | Caché actualizada.                   |

## Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
from functools import lru_cache
import time

@lru_cache(maxsize=None)
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

t0 = time.perf_counter()
res = fibonacci(50)
t1 = time.perf_counter()

print(f"Fibonacci(50) = {res}")
print(f"Calculado en: {(t1 - t0)*1000:.4f} ms (Tiempo O(n))")
```

### ⚠ Gotcha Frecuente

Recursiones muy profundas superan el límite de la pila de llamadas (por defecto 1000).

### ❌ Antipatrón

```
def contar(n): if n == 0: return 0 return contar(n - 1)
# ❌ Falla con n > 1000 por RecursionError
```

### ✅ Patrón Correcto

```
# Enfoque iterativo (Bottom-Up) o sys.setrecursionlimit
def contar_iterativo(n): return sum(range(n)) # ✅ O(1) memoria
```

## Conclusiones Generales del Curso

Has completado las 8 semanas de **Curso 2: Algoritmos Avanzados y Estructuras de Datos**.



### Certificación del Nivel

Dominio conceptual y práctico consolidado.

## Bibliografía Canónica

| Recurso                      | Descripción             | Enlace                                                                    |
|------------------------------|-------------------------|---------------------------------------------------------------------------|
| Documentación Oficial Python | Especificación canónica | <a href="https://docs.python.org/3/">docs.python.org/3/</a>               |
| PEP 8                        | Guía de estilo          | <a href="https://peps.python.org/pep-0008/">peps.python.org/pep-0008/</a> |
| wisrovi SUITE                | Librerías open source   | <a href="https://github.com/wisrovi">github.com/wisrovi</a>               |